

Project Report

FRAUDGUARD AI

Credit Card Fraud Detection Using Machine Learning

Submitted by

Mugdha Devidas Bramhankar

Registration Number: 25MIB10002

AI/ML Project

1. Introduction

Credit card transactions have become a common part of everyday life. People use credit and debit cards for shopping, online payments, subscriptions, and many other activities. As the number of digital transactions increases, the possibility of fraudulent or unauthorized transactions also becomes an important concern.

Checking every transaction manually is not practical when there are thousands or millions of transactions. Machine learning can help by learning patterns from previously available transaction data and using those patterns to identify transactions that may be fraudulent.

FraudGuard AI is a machine learning-based credit card fraud detection project developed to demonstrate how artificial intelligence can be applied to this problem. The project takes transaction data, performs basic data analysis and preprocessing, trains a machine learning model, and provides predictions through a simple Streamlit web application.

The project is mainly developed for educational and demonstration purposes. It shows the complete flow of a basic machine learning project, from dataset analysis to model prediction and user interaction.

2. Problem Statement

Credit card fraud can result in financial loss and inconvenience for customers and financial institutions. A large transaction dataset can contain a very small number of fraudulent transactions compared with normal transactions, making fraud detection a challenging classification problem.

The main problem addressed by this project is:

How can machine learning be used to identify whether a credit card transaction is normal or potentially fraudulent based on transaction data?

The project aims to build a simple system that can analyze transaction information and provide a prediction of whether a transaction belongs to the normal or fraudulent category.

3. Objectives of the Project

The main objectives of FraudGuard AI are:

- To understand the application of machine learning in fraud detection.
 - To analyze a credit card transaction dataset.
 - To identify the distribution of normal and fraudulent transactions.
 - To preprocess the data before model training.
 - To train a machine learning classification model.
 - To save the trained model for future predictions.
 - To create a simple prediction system.
 - To develop an interactive Streamlit dashboard.
 - To display useful transaction statistics and visualizations.
 - To understand the complete workflow of an AI/ML project.
-

4. Scope of the Project

The scope of this project is focused on demonstrating a basic machine learning approach for credit card fraud detection.

The project includes:

- Dataset loading and checking.
- Data analysis.
- Data preprocessing.
- Machine learning model training.
- Saving the trained model.
- Loading the model for prediction.
- Predicting normal and potentially fraudulent transactions.
- Displaying transaction statistics.
- Creating an interactive Streamlit interface.

The project does not attempt to replace a real banking fraud detection system. Real-world financial systems would require much larger datasets, advanced fraud detection techniques, security controls, real-time processing, monitoring, and regular model updates.

5. Functional Requirements

The functional requirements describe what the system should do.

5.1 Dataset Loading

The system should be able to load the credit card transaction dataset from the project data folder.

5.2 Data Checking

The system should check the dataset and provide basic information such as the number of records, columns, and target classes.

5.3 Data Analysis

The system should analyze the transaction dataset and show the distribution between normal and fraudulent transactions.

5.4 Data Preprocessing

The system should prepare the transaction data so that it can be used by the machine learning model.

5.5 Model Training

The system should train a classification model using the available transaction data.

5.6 Model Saving

The trained model should be saved so that it can be reused without training the model every time.

5.7 Transaction Prediction

The system should accept transaction-related input and provide a prediction indicating whether the transaction is classified as normal or potentially fraudulent.

5.8 Dashboard

The Streamlit application should display important information about the dataset and provide an easy-to-use interface.

6. Non-Functional Requirements

The non-functional requirements describe the general qualities expected from the project.

6.1 Usability

The application should be simple enough for a student or beginner to understand and use.

6.2 Performance

The application should load the dataset and provide predictions within a reasonable amount of time.

6.3 Maintainability

The project should be divided into separate Python files so that individual parts can be modified easily.

6.4 Reliability

The application should provide consistent results when the same valid input is provided.

6.5 Portability

The project should be able to run on a computer with Python and the required libraries installed.

6.6 Security

The project should not be treated as a production banking system. Sensitive real-world customer information should not be used in the application.

Main Use Cases

- Open the application.
- View transaction statistics.
- View normal/fraud distribution.
- Enter transaction information.
- Generate a prediction.
- View the prediction result.

Design Decisions and Rationale

Python

Pandas

Pandas is used for loading, processing, and analyzing the transaction dataset.

Scikit-learn

Scikit-learn provides tools for preprocessing, machine learning, model training, and evaluation. It supports classification and preprocessing tasks required for this project.

Streamlit

Streamlit was selected for the user interface because it allows Python-based data and machine learning applications to be turned into interactive web applications without requiring a separate frontend framework.

Modular Project Structure

Different tasks are kept in separate Python files. This makes the project easier to understand, test, and modify.

Technologies Used

The main technologies and tools used in this project are:

Technology	Purpose
Python	Main programming language
Pandas	Dataset handling and analysis
NumPy	Numerical operations

Technology	Purpose
Scikit-learn	Machine learning and preprocessing
Streamlit	Web application/dashboard
Matplotlib/Charts	Data visualization
Pickle	Saving trained model objects
VS Code	Development environment
Git	Version control
GitHub	Project repository

Implementation Details

The project is divided into different files, with each file having a specific purpose.

`check_data.py`

This script is used to check the dataset and understand its basic structure.

It helps verify:

- Dataset loading.
- Number of records.
- Available columns.
- Basic dataset information.
- Target class information.

`analysis.py`

The analysis script is used to explore the dataset.

It helps understand:

- Number of normal transactions.
- Number of fraudulent transactions.
- Distribution of transaction classes.
- Basic patterns in the data.

This step is important because understanding the dataset before training a model helps identify possible data-related issues.

`train_model.py`

This file handles the machine learning training process.

The general process is:

```
Load Dataset
  |
  v
Separate Features and Target
  |
  v
Preprocess Data
  |
  v
Train Machine Learning Model
  |
  v
Save Model
```

The trained model is stored in:

`src/models/fraud_model.pkl`

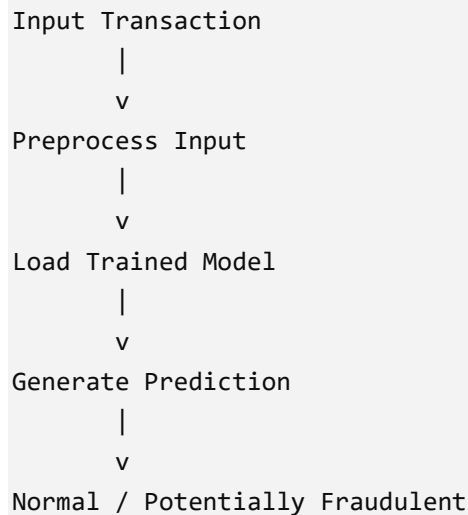
The preprocessing/scaling object is stored in:

`src/models/scaler.pkl`

`predict.py`

The prediction script is responsible for loading the trained model and using it to classify transaction input.

The basic process is:



`app.py`

The `app.py` file contains the Streamlit application.

It provides an interactive interface where users can view information about the dataset and interact with the fraud detection system.

The application displays transaction statistics and visualizations and provides the project with a simple web-based interface.

Streamlit applications can be run using the `streamlit run` command.

Dataset

The project uses a credit card transaction dataset containing transaction-related features and a target class.

The target column is:

Class

The two main classes are:

0 = Normal Transaction
1 = Fraudulent Transaction

The dataset used in the project contains:

- **Total transactions:** 284,807
- **Normal transactions:** 284,315
- **Fraudulent transactions:** 492

One important observation from the dataset is that fraudulent transactions form a very small portion of all transactions. This makes fraud detection an interesting machine learning problem because the model needs to identify the less common class rather than simply relying on the majority class.

Results and Output

After running the analysis, the project successfully displayed the transaction distribution.

Dataset Summary

Category	Number of Transactions
Total	284,807
Normal	284,315
Fraudulent	492

The Streamlit dashboard displays the main transaction statistics in an easy-to-understand format.

The application also provides a visual representation of the transaction distribution.

Result

The developed system successfully:

- Loaded the transaction dataset.
- Checked the dataset.
- Performed basic data analysis.
- Trained a machine learning model.
- Saved the trained model.
- Loaded the model for prediction.
- Created a Streamlit dashboard.
- Displayed transaction statistics.
- Provided a basic fraud prediction

Testing Approach

Testing was performed at different stages of the project to make sure that the main components were working correctly.

Test	Expected Result	Status
Load dataset	Dataset loads successfully	Passed
Check dataset	Dataset information is displayed	Passed
Analyze classes	Normal/fraud counts are displayed	Passed
Train model	Model is trained successfully	Passed
Save model	Model file is created	Passed
Load model	Saved model loads successfully	Passed
Run prediction	Prediction is generated	Passed
Start Streamlit app	Dashboard opens successfully	Passed
Display statistics	Transaction statistics appear	Passed

Testing was mainly focused on checking whether each part of the project works correctly and whether the complete application can be executed from the project folder.

Challenges Faced

During the development of the project, several challenges were encountered.

Understanding the Dataset

The dataset contains many columns and a large number of transactions. Understanding the structure of the data was one of the first challenges.

Imbalanced Data

There is a large difference between the number of normal and fraudulent transactions. Only a small number of transactions are fraudulent compared with the total dataset.

This makes fraud detection more challenging because the model needs to identify the minority class.

Data Preprocessing

The transaction data needs to be prepared correctly before being given to the machine learning model. Scaling and maintaining the same preprocessing approach during prediction are important parts of the workflow.

Connecting the Model with the Application

Another challenge was connecting the trained machine learning model with the Streamlit interface so that the application could use the saved model to generate predictions.

Project Organization

Keeping the dataset, Python scripts, saved models, and documentation organized was also important for making the project easier to understand and submit.

Learnings and Key Takeaways

This project provided practical experience in several areas of AI and machine learning.

The main things learned from this project are:

- How to work with a real-world-style dataset.
- How to inspect and analyze data using Python.
- How machine learning classification can be applied to fraud detection.
- Why data preprocessing is important.
- How to train and save a machine learning model.
- How to use a saved model for prediction.
- How to create a basic interactive ML application using Streamlit.
- How to organize an AI/ML project into multiple files.
- How Git and GitHub can be used for project management and submission.
- How to document the complete development process.

One of the biggest takeaways from this project was understanding that building an AI/ML application involves more than just training a model. Data preparation, testing, user interface, documentation, and project organization are also important.

Future Enhancements

The current project can be improved in several ways in the future.

Improved Machine Learning Models

Different classification algorithms can be tested and compared to find an approach suitable for the dataset.

Better Evaluation

More evaluation metrics such as precision, recall, F1-score, confusion matrix, and ROC-AUC can be included to provide a better understanding of model performance.

Real-Time Detection

The system could be extended to process transactions in real ti **Improved Dashboard**

The dashboard could include additional charts, filters, model performance information, and more detailed transaction analysis.

Database Integration

A database could be added instead of relying only on a CSV file.

Model Monitoring

A future version could monitor model performance over time and retrain the model when transaction patterns change.

Deployment

The Streamlit application could be deployed online so that it can be accessed remotely.

Limitations

Although FraudGuard AI demonstrates the basic concept of machine learning-based fraud detection, it has some limitations.

- It is an educational project and not a production banking system.
 - The model depends on the dataset used for training.
 - Real-world transaction patterns can change over time.
 - The project does not include a complete banking security system.
 - It does not connect to a real credit card or banking network.
 - The prediction should not be treated as a final financial decision.
 - More extensive testing would be required before using a system like this in a real-world environment.
-

Conclusion

FraudGuard AI demonstrates how machine learning can be used to approach the problem of credit card fraud detection.

The project starts with a transaction dataset and goes through the major stages of an AI/ML workflow: data checking, analysis, preprocessing, model training, model saving, prediction, and application development.

A Streamlit dashboard was also developed to make the project easier to interact with and understand. The dashboard displays transaction statistics and provides a simple interface for the fraud detection system.

Through this project, practical knowledge was gained about Python, data analysis, machine learning, model handling, Streamlit, testing, and project documentation.

Overall, the project provides a useful learning example of how an AI/ML concept can be converted into a working application.

References

1. **Scikit-learn Documentation** – Machine learning and preprocessing tools for Python.
 2. **Streamlit Documentation** – Documentation for developing interactive Python-based data applications.
 3. **Python Documentation** – Python programming language documentation.
 4. **Kaggle Credit Card Fraud Detection Dataset** – Dataset used for the project.
 5. **Project source code and implementation** – FraudGuard AI GitHub repository.
-

Project Structure

The final project is organized approximately as follows:

CREDIT-CARD-FRAUD-DETECTION/

```
|
|├── data/
|│   └── creditcard.csv
|
|├── src/
|│   ├── app.py
|│   ├── analysis.py
|│   ├── check_data.py
|│   ├── predict.py
|│   ├── train_model.py
|│   |
|│   └── models/
|│       ├── fraud_model.pkl
|│       └── scaler.pkl
|
|├── README.md
|├── statement.md
|├── requirements.txt
|└── .gitignore
```

Final Project Summary

Project Name: FraudGuard AI

Project Type: AI/ML Project

Domain: Machine Learning / Fraud Detection

Language: Python

Interface: Streamlit

Dataset: Credit Card Transaction Dataset

Target: Normal or Fraudulent Transaction

Student: Mugdha Devidas Bramhankar

Registration Number: 25MIB10002

Project objective: To demonstrate the use of machine learning for identifying potentially fraudulent credit card transactions and presenting the results through an interactive application.